

Assignment On Operation Research

SOBURUL ISLAM
B200304003

Submitted on 10 May 2026

Answer to the problem no 1

Let the quantity (units) of food types is x_i . Then, the decision variables are x_1 , x_2 and x_3 for food types F_1 , F_2 and F_3 respectively. Given that, the cost of food types 50 TK., 70 TK. and 60 TK. per unit. Since, dietician wants to minimize the cost, so the objective function is minimum.

Objective function,

$$\text{Minimize, } Z = 50x_1 + 70x_2 + 60x_3$$

Subject to,

$$4x_1 + 5x_2 + 3x_3 \geq 6$$

$$6x_1 + 3x_2 + 5x_3 \geq 9$$

$$2x_1 + 3x_2 + 4x_3 \geq 4$$

$$x_1, x_2, x_3 \geq 0$$

To solve all the linear programming problems, we have to use the lpSolve package in R.

```
library(lpSolve)

## Warning: package 'lpSolve' was built under R version 4.3.3

# coefficients of the objective function
obj <- c(50, 70, 60)

# coefficients matrix of the constraints
const <- matrix(c(
  4, 5, 3, # For vitamin A
  6, 3, 5, # For vitamin B
  2, 3, 4 # For vitamin C
), nrow = 3, byrow = TRUE)

# direction of the constraints
dir <- c(">=", ">=", ">=")

# right hand side of the constraints
rhs <- c(6, 9, 4)

# Solution
sol <- lp(
  direction = "min",
  objective.in = obj,
  const.mat = const,
  const.dir = dir,
  const.rhs = rhs
)
```

```
cat("The optimal values are:", "\n", "x1 = ", sol$solution[1], "\n",
    "x2 = ", sol$solution[2], "\n", "x3 = ", sol$solution[3], "\n")
cat("Minimum cost of the food mixture = ", sol$objval, "TK.", "\n")
```

```
## The optimal values are:
## x1 = 1.2
## x2 = 0
## x3 = 0.4
## Minimum cost of the food mixture = 84 TK.
```

Answer to the problem no 2

In **Phase-I**, we can write the given LP model by adding surplus and artificial variable,

Objective function,

$$\text{Minimize, } Z = 0x_1 + 0x_2 + 0e_1 + 0e_2 + a_1 + a_2$$

Subject to,

$$2x_1 + 4x_2 - e_1 + 0e_2 + a_1 + 0a_2 = 4$$

$$x_1 + 7x_2 + 0e_1 - e_2 + 0a_1 + a_2 = 7$$

$$x_1, x_2, e_1, e_2, a_1, a_2 \geq 0$$

```
# coefficients of the objective function
```

```
obj1 <- c(0,0,0,0,1,1)
```

```
# coefficients matrix of the constraints
```

```
const1 <- matrix(c(
    2, 4, -1, 0, 1, 0,
    1, 7, 0, -1, 0, 1
), nrow = 2, byrow = TRUE)
```

```
# direction of the constraints
```

```
dir1 <- c("=", "=")
```

```
# right hand side of the constraints
```

```
rhs1 <- c(4,7)
```

```
# Solution
```

```
phase1 <- lp(
    direction = "min",
    objective.in = obj1,
    const.mat = const1,
    const.dir = dir1,
    const.rhs = rhs1
)
cat("After minimizing the artificial variables, the optimal value = ", phase1$objval, "\n")
cat("The values of the artificial variable are:", "\n",
    "a1 = ", phase1$solution[5], " and ", "a2 = ", phase1$solution[6], "\n")
```

```
## After minimizing the artificial variables, the optimal value = 0
## The values of the artificial variable are:
## a1 = 0 and a2 = 0
```

Since the optimal value in Phase-I is zero also artificial variables are zero, so the original LP has feasible solution. Now, we can generate **Phase-II** by changing the Phase-I objective function by the original objective function $Z = x_1 + x_2$.

Minimize $Z = x_1 + x_2 + 0e_1 + 0e_2$
Subject to:

$$2x_1 + 4x_2 - e_1 + 0e_2 = 4$$

$$x_1 + 7x_2 + 0e_1 - e_2 = 7$$

$$x_1, x_2, e_1, e_2 \geq 0$$

```
# coefficients of the objective function
obj2 <- c(1, 1, 0, 0)

# coefficients matrix of the constraints
const2 <- matrix(c(
  2, 4, -1, 0,
  1, 7, 0, -1
), nrow = 2, byrow = TRUE)

# direction of the constraints
dir2 <- c("=", "=")

# right hand side of the constraints
rhs2 <- c(4, 7)

# Solution
phase2 <- lp(
  direction = "min",
  objective.in = obj2,
  const.mat = const2,
  const.dir = dir2,
  const.rhs = rhs2
)
cat("The optimal values are:", "\n",
    "x1 = ", phase2$solution[1], "\n", "x2 = ", phase2$solution[2], "\n")
cat("Optimal value of Z = ", phase2$objval, "\n")

## The optimal values are:
## x1 = 0
## x2 = 1
## Optimal value of Z = 1
```

Answer to the problem no 3

Let the products P_1 and P_2 are denoted by x_1 and x_2 .

Given that, the profit per unit 40 TK. and 30 TK. for P_1 and P_2 . Since, company wants to maximize the profit, so the objective function is maximum.

Objective function,

$$\text{Maximize, } Z = 40x_1 + 30x_2$$

Subject to,

$$2x_1 + x_2 \leq 100$$

$$x_1 + 2x_2 \leq 80$$

$$x_1, x_2 \geq 0$$

To solve all the linear programming problems, we have to use the lpSolve package in R.

```
library(lpSolve)
# coefficients of the objective function
obj <- c(40, 30)
```

```

# coefficients matrix of the constraints
const <- matrix(c(
    2, 1,  # For machine 1
    1, 2   # For machine 2
), nrow = 2, byrow = TRUE)

# direction of the constraints
dir <- c("<=", "<=")

# right hand side of the constraints
rhs <- c(100, 80)

# Solution
sol <- lp(
    direction = "max",
    objective.in = obj,
    const.mat = const,
    const.dir = dir,
    const.rhs = rhs
)

cat("The optimal values are:", "\n", "x1 = ", sol$solution[1], "\n",
    "x2 = ", sol$solution[2], "\n")
cat("Maximum profit for given constraints = ", sol$objval, "TK.", "\n")

## The optimal values are:
## x1 = 40
## x2 = 20
## Maximum profit for given constraints = 2200 TK.

```

Answer to the problem no 4

We can write the given LPP introducing slack, surplus, artificial variables as,
Objective function,

$$\text{Minimize, } Z = 5x_1 + 3x_2 + 0s_1 + 0e_1 + Ma_1 + Ma_2$$

Subject to,

$$2x_1 + 4x_2 + s_1 + 0e_1 + 0a_1 + 0a_2 = 12$$

$$2x_1 + 2x_2 + 0s_1 + 0e_1 + a_1 + 0a_2 = 10$$

$$5x_1 + 2x_2 + 0s_1 - e_1 + 0a_1 + a_2 = 10$$

$$x_1, x_2, s_1, e_1, a_1, a_2 \geq 0$$

To solve all the linear programming problems, we have to use the lpSolve package in R.

```

library(lpSolve)
M <- 1e7

# coefficients of the objective function
obj <- c(5, 3, 0, 0, M, M)

# coefficients matrix of the constraints
const <- matrix(c(
    2, 4, 1, 0, 0, 0,
    2, 2, 0, 0, 1, 0,
    5, 2, 0, -1, 0, 1

```

```

    ), nrow = 3, byrow = TRUE)

# direction of the constraints
dir <- c("=", "=", "=")

# right hand side of the constraints
rhs <- c(12, 10, 10)

# Solution
sol <- lp(
  direction = "min",
  objective.in = obj,
  const.mat = const,
  const.dir = dir,
  const.rhs = rhs
)

cat("The optimal values are:", "\n", "x1 = ", sol$solution[1], "\n",
    "x2 = ", sol$solution[2], "\n")
cat("Optimal value of Z = ", sol$objval, "\n")

## The optimal values are:
## x1 = 4
## x2 = 1
## Optimal value of Z = 23

```

Operation Research

Example-1: A garment manufacturer has a production line making two styles of shirts. Style-I needs 100g of cotton thread, 200g of Dacron thread and 200g of linen thread. Corresponding requirements of style-II are 100g, 300g and 200g. The net contributions are 100 Taka for style-I and 120 Taka for style-II. The available inventory of cotton thread, Dacron thread and linen thread are, respectively, 34kg, 36kg and 32kg. The manufacturer wants to determine the number of each style to be produced with the given inventory. Formulate the LPP model and solve it by graphical method using R code.

Solution: Let x and y be the number of style-I and style-II shirts produces. Given that, net contributions are 100 Taka for style-I and 120 Taka for style-II and producer wants to maximize profit by producing more shirts.

The objective function, maximize $z = 100x + 120y$.

For the given information constraints,

$$\begin{aligned} 100x + 100y &\leq 34000 \\ 200x + 300y &\leq 36000 \\ 200x + 200y &\leq 32000 \\ x \geq 0, y &\geq 0 \end{aligned}$$

Graphically drawing the lines;

```
x<-seq(0,400,1)
yc<-(34000-100*x)/100
yd<-(36000-200*x)/300
yl<-(32000-200*x)/200
plot(x, yc, type="n",xlab="x",ylab="y",
      xlim=c(0,400), ylim=c(0,400))
lines(x,yc,col="red")
lines(x,yd,col="blue")
lines(x,yl,col="green")
```

The corner points are: (0, 120), (160, 0), (120, 40).

Calculating the objective value for all corner points;

```
df<-data.frame(x=c(0,160,120),y=c(120,0,40))
df$z<-100*df$x+120*df$y
opt<-which.max(df$z)
xy<-df[opt,]
xy
  x  y  z
120 40 16800
```

Hence, to maximize the profit the producer should produce style-I 120 and style-II 40 units.

Example-2: An animal feed company must produce 200 kg of a mixture consisting of ingredients A and B daily. A costs 3 per kg and B costs 8 per kg. Not more than 80 kg of A can be used and at least 60 kg of B must be used. The company wants to know how much of each ingredient should be used to minimize cost. Formulate the LPP.

Solution: Let x and y denotes the quantities (kg) of ingredients A and B. Given that, the costs 3 and 8 per kg of ingredient A and B. The company want to minimize the mixture cost.

The objective function, minimize $z = 3x + 8y$.

For the given information constraints,

$$x + y = 200$$

$$x \leq 80$$

$$y \geq 60$$

$$x \geq 0, y \geq 0$$

Graphically drawing the lines;

```
x<-seq(0,220,1)
y1<-200-x
y2<-80-0*x
y3<-60-0*x
plot(x, y1, type="n",xlab="x",ylab="y",
     xlim=c(0,220), ylim=c(0,220))
lines(x,y1,col="red")
lines(y2,x,col="blue")
lines(x,y3,col="green")
```

The corner points are: (80, 120), (0, 200). For constraint $x + y = 200$, corner points (0, 60) and (80, 60) are not feasible.

Calculating the objective value for all corner points;

```
df<-data.frame(x=c(80,0),y=c(120,200))
df$z<-3*df$x+8*df$y
opt<-which.min(df$z)
xy<-df[opt,]
xy
  x  y  z
80 120 1200
```

Hence, to minimize the total cost, the company should use 80 kg of ingredient A and 120 kg of ingredient B.

Example-3: A factory has engaged in manufacturing three products A, B and C which involve lathe work, grinding and assembling. The cutting, grinding and assembling times required for one unit of A are 2, 1 and 1 hour respectively. Similarly, they were 3, 1 and 3 hours for one unit of B and 1, 3 and 1 hours for one unit of C. The profits on A, B and C are TK.2, TK.2 and TK. 4 per unit respectively. Assuming that there are

available 300 hours of lathe time, 300 hours of grinder time and 240 hours of assembly time. Formulate the problem mathematically. How many units of each product should be produced to maximize profit?

Solution: Let x_1 = number of units of product A produced, x_2 = number of units of product B produced, x_3 = number of units of product C produced.

Since the profits are TK. 2, TK. 2, and TK. 4 per unit for products A, B, and C, respectively, the factory aims to maximize its total profit.

The objective function, maximize $z = 2x_1 + 2x_2 + 4x_3$.

For the given information constraints,

$$2x_1 + 3x_2 + x_3 \leq 300$$

$$x_1 + x_2 + 3x_3 \leq 300$$

$$x_1 + 3x_2 + x_3 \leq 240$$

$$x_1 \geq 0, x_2 \geq 0, x_3 \geq 0$$

Since, there are more two variables and less or equal constraints, to solve this linear programming problems we use simplex method.

```
library(lpSolve)
obj<-c(2,2,4)
const<-matrix(c(2,1,1,3,1,3,1,3,1),nrow=3)
dir<-c("<=", "<=", "<=")
rhs<-c(300,300,240)
sol<-lp(
  direction="max",
  objective.in=obj,
  const.mat=const,
  const.dir=dir,
  const.rhs=rhs)
cat("The optimal values are:", "\n", "x1=", sol$solution[1], "\n",
    "x2=", sol$solution[2], "\n", "x3=", sol$solution[3], "\n")
The optimal values are:
  x1 = 120
  x2 = 0
  x3 = 60
cat("Maximum Profit =", sol$objval, "TK.", "\n")
Maximum Profit = 480 TK.
```

Example-4: Solve the following LPP using R code by Simplex method,

$$\text{Minimize, } z = x_1 + x_2 - 4x_3$$

Subject to,

$$x_1 + x_2 + 2x_3 \leq 8$$

$$x_1 + x_2 - x_3 \leq 2$$

$$-x_1 + x_2 + x_3 \leq 4$$

$$x_1, x_2, x_3 \geq 0$$

Solution: Using lpSolve package in R;

```
library(lpSolve)
obj<-c(1,1,-4)
const<-matrix(c(1,1,-1,1,1,1,2,-1,1),nrow=3)
dir<-c("<=", "<=", "<=")
rhs<-c(8,2,4)
sol<-lp(
  direction="min",
  objective.in=obj,
  const.mat=const,
  const.dir=dir,
  const.rhs=rhs)
cat("The optimal values are:", "\n", "x1=", sol$solution[1], "\n",
    "x2=", sol$solution[2], "\n", "x3=", sol$solution[3], "\n")
The optimal values are:
x1 = 0
x2 = 0
x3 = 4
cat("Minimum value =", sol$objval, "\n")
Minimum value = -16
```

Example-5: Solve the following LPP using R code by dual simplex method,

$$\text{Minimize, } z = 5x_1 + 6x_2$$

Subject to,

$$x_1 + x_2 \geq 2$$

$$4x_1 + x_2 \geq 4$$

$$x_1, x_2 \geq 0$$

Solution: To solve the given LPP by dual simplex method we have to maximize the objective function and have to change the constraint into less than or equal.

$$\text{Maximize, } z = -5x_1 - 6x_2$$

Subject to,

$$-x_1 - x_2 \leq -2$$

$$-4x_1 - x_2 \leq -4$$

$$x_1, x_2 \geq 0$$

Using lpSolve package in R;

```
library(lpSolve)
obj<-c(-5,-6)
const<-matrix(c(-1,-4,-1,-1),nrow=2)
dir<-c("<=", "<=")
```

```

rhs<-c(-2,-4)
sol<-lp(
  direction="max",
  objective.in=obj,
  const.mat=const,
  const.dir=dir,
  const.rhs=rhs)
cat("The optimal values are:", "\n", "x1=", sol$solution[1], "\n",
    "x2=", sol$solution[2], "\n")
The optimal values are:
  x1 = 2
  x2 = 0
cat("Maximum value =", sol$objval, "\n")
Maximum value = -10

```

Hence, the minimum objective value = $-(\text{maximum}) = -(-10) = 10$.

Example-6: Solve the following LPP using R code by Big-M method,

Minimize, $z = 3x_1 + 2x_2$

Subject to,

$$2x_1 + x_2 \geq 10$$

$$-3x_1 + 2x_2 \leq 6$$

$$x_1 + x_2 \geq 6$$

$$x_1, x_2 \geq 0$$

Solution: To solve the given LPP by Big-M method, we write the given LPP introducing slack, surplus and artificial variables as,

Minimize, $z = 3x_1 + 2x_2 + 0s_1 + 0e_1 + 0e_2 + Ma_1 + Ma_2$

Subject to,

$$2x_1 + x_2 + 0s_1 - e_1 + 0e_2 + a_1 + 0a_2 = 10$$

$$-3x_1 + 2x_2 + s_1 + 0e_1 + 0e_2 + 0a_1 + 0a_2 = 6$$

$$x_1 + x_2 + 0s_1 + 0e_1 - e_2 + 0a_1 + a_2 = 6$$

$$x_1, x_2, s_1, e_1, e_2, a_1, a_2 \geq 0$$

Using lpSolve package in R;

```

library(lpSolve)
M<-1e10
obj<-c(3,2,0,0,0,M,M)
const<-matrix(c(2,-3,1,1,2,1,0,1,0,-1,0,0,0,0,-1,1,0,0,0,0,1),nrow=3)
dir<-c("=", "=", "=")
rhs<-c(10,6,6)
sol<-lp(
  direction="min",
  objective.in=obj,
  const.mat=const,
  const.dir=dir,
  const.rhs=rhs)

```

```
cat("The optimal values are:", "\n", "x1=", sol$solution[1], "\n",
    "x2=", sol$solution[2], "\n")
The optimal values are:
x1 = 4
x2 = 2
cat("Minimum value =", sol$objval, "\n")
Minimum value = 16
```

Example-7: Solve the following LPP using R code by two-phase method,

Minimize, $z = 2x_1 + 3x_2$

Subject to,

$$2x_1 + x_2 \leq 16$$

$$x_1 + 3x_2 \geq 36$$

$$x_1 + x_2 = 10$$

$$x_1, x_2 \geq 0$$

Solution: In Phase-I, we can write the given LP model by adding surplus and artificial variable as,

Minimize, $z = 0x_1 + 0x_2 + 0s_1 + 0e_1 + a_1 + a_2$

Subject to,

$$2x_1 + x_2 + s_1 + 0e_1 + 0a_1 + 0a_2 = 16$$

$$x_1 + 3x_2 + 0s_1 - e_1 + a_1 + 0a_2 = 36$$

$$x_1 + x_2 + 0s_1 + 0e_1 + 0a_1 + a_2 = 10$$

$$x_1, x_2, s_1, e_1, e_2, a_1, a_2 \geq 0$$

```
library(lpSolve)
obj1<-c(0,0,0,0,1,1)
const1<-matrix(c(2,1,1,1,3,1,1,0,0,0,-1,0,0,1,0,0,0,1),nrow=3)
dir1<-c("=", "=", "=")
rhs1<-c(16,36,10)
phase1<-lp(
  direction="min",
  objective.in=obj1,
  const.mat=const1,
  const.dir=dir1,
  const.rhs=rhs1)
cat("After minimizing the artificial variables, the optimal value =", phase1$objval, "\n")
After minimizing the artificial variables, the optimal value = 6
cat("The values of the artificial variables are:", "\n",
    "a1=", phase1$solution[5], " and ", "a2=", phase1$solution[6], "\n")
The values of the artificial variables are:
a1= 6 and a2= 0
```

Since the optimal value in Phase-I is 6 also artificial variables are not zero, so the original LP has no feasible solution.

Example-8: Find the optimum integer solution to the following LPP using R code,

Maximize, $z = x_1 + x_2$

Subject to,

$$3x_1 + 2x_2 \leq 5$$

$$x_2 \leq 2$$

$$x_1, x_2 \geq 0 \text{ and integers}$$

Solution: Using lpSolve package in R;

```
library(lpSolve)
obj<-c(1,1)
const<-matrix(c(3,0,2,1),nrow=2)
dir<-c("<=", "<=")
rhs<-c(5,2)
sol<-lp(
  direction="max",
  objective.in=obj,
  const.mat=const,
  const.dir=dir,
  const.rhs=rhs,
  all.int=TRUE)
cat("The optimal values are:", "\n", "x1, x2 = ", sol$solution[1:2], "\n")
  x1, x2 = 1 1
cat("Maximum value = ", sol$objval, "\n")
  Maximum value = 2
```

Example-9: Find the optimal solution for the following TP using R code,

Destination Source	D_1	D_2	D_3	Supply
S_1	4	8	8	76
S_2	16	24	16	82
S_3	8	16	24	77
Demand	72	102	41	

Solution: Using lpSolve package in R;

```
library(lpSolve)
cost<-matrix(c(4,16,8,8,24,16,8,16,24),nrow=3)
rsig<-rep("<",3)
rrhs<-c(76,82,77)
csig<-rep(">",3)
crhs<-c(72,102,41)
sol<-lp.transport(cost,"min",rsig,rrhs,csig,crhs)
sol
```

Success: the objective function is 2424

sol\$solution

	[,1]	[,2]	[,3]
[1,]	0	76	0
[2,]	21	0	41
[3,]	51	26	0

Operation Research and Biostatistics Lab

Mohammad Sobrul
Department of Statistics
Jagannath University, Dhaka-1100

Chapter 2

Biostatistics

Example-1: If a product failure time (in hours) is assumed to follow an exponential distribution with the failure rate $\lambda = 0.00025$, then answer the following questions:

- (a) What is the probability that the product will survive 10,000 hours?
- (b) What is the probability that the product will survive the next 20,000 hours?
- (c) What is the mean time to failure (MTTF)?
- (d) What is the median time to failure?
- (e) At what point in time is it expected that 30% of the products will fail?
- (f) When will 63.21% fail?

Solution: The probability density function of exponential distribution with the failure rate $\lambda = 0.00025$ is

$$f(t; \lambda) = \lambda e^{-\lambda t} \quad ; \quad t > 0, \lambda > 0$$

Hence, the survival function and the hazard function

$$S(t) = e^{-\lambda t} ; t > 0 \quad \text{and} \quad h(t) = \lambda ; t > 0$$

- a.** The probability of surviving 10,000 hours is 0.082085.

```
lambda <- 0.00025  
St1 <- exp(-lambda*10000)
```

- b.** The probability that the product will survive the next 20,000 hours is 0.006737947.

$$S(t_2) = P(T > t_1 + t_2 | T > t_1) = \frac{S(t_1 + t_2)}{S(t_1)} = \frac{e^{-\lambda(t_1+t_2)}}{e^{-\lambda t_1}} = e^{-\lambda t_2}$$

```
St2 <- exp(-lambda*20000)
```

- c.** The mean time to failure (MTTF) is 4000. MTTF is the expected lifetime from the beginning until failure, i.e., $E(T)$.

```
MTTF <- 1/lambda
```

- d.** The median survival time is 2772.589.

```
T50 <- (1/lambda)*log(2)
```

- e.** 30% of the products will fail means 70% product will survive, i.e., at 1426.7 hours' time it expected that 30% of the products will fail.

```
T70 <- -1/lambda*log(0.70)
```

- f.** 63.21% of the products will fail means 36.79% product will survive, i.e., at 3999.776 hours' time it expected that 63.21% of the products will fail.

```
T36 <- -1/lambda*log(1-0.6321)
```

Example-2: If a product failure time (in hours) is assumed to follow a Weibull distribution with scale parameter $\lambda = 4000$ and shape parameter $\alpha = 1.50$, then find:

- What are the mean and variance of time to failure?
- What is the probability that the product will operate for 5000 h?
- What is the probability that the product will operate for another 2000 h after operating for 5000 h?

Solution: The probability density function of Weibull distribution is

$$f(t) = \frac{\alpha}{\lambda} \left(\frac{t}{\lambda}\right)^{\alpha-1} e^{-\left(\frac{t}{\lambda}\right)^\alpha}; t \geq 0 \quad \text{-----}(1)$$

Where, $\lambda = \text{Scale parameter}$ and $\alpha = \text{Shape parameter}$.

Another Form:

$$f(t) = \alpha\lambda(\lambda t)^{\alpha-1} e^{-(\lambda t)^\alpha}; t \geq 0 \quad \text{-----}(2)$$

Where, $\lambda = \text{Rate parameter}$ and $\alpha = \text{Shape parameter}$.

The mean and variance of (1) is

$$E(T) = \lambda \Gamma\left(\frac{1}{\alpha} + 1\right) \text{ and } V(T) = \lambda^2 \left[\Gamma\left(\frac{2}{\alpha} + 1\right) - \left\{ \Gamma\left(\frac{1}{\alpha} + 1\right) \right\}^2 \right]$$

The mean and variance of (2) is

$$E(T) = \frac{1}{\lambda} \Gamma\left(\frac{1}{\alpha} + 1\right) \text{ and } V(T) = \frac{1}{\lambda^2} \left[\Gamma\left(\frac{2}{\alpha} + 1\right) - \left\{ \Gamma\left(\frac{1}{\alpha} + 1\right) \right\}^2 \right]$$

- a.** The mean and variance of time to failure is 3610.981 and 6011045.

```
a <- 4000
la <- 1.50
mean.T <- a*gamma(1/la+1)
var.T <- la^2*(gamma(2/la+1)-(gamma(1/la+1))^2)
```

- b.** The probability that the product will operate for 5000 h is 0.2472037.

$$S(t) = e^{-\left(\frac{t}{\lambda}\right)^\alpha}$$

```
St5 <- exp(-(5000/la)^a)
```

- c.** The probability that the product will operate for another 2000 h after operating for 5000 h is 0.3995206.

$$S(t_2) = P(T > t_5 + t_2 | T > t_5) = S(t_5 + t_2) / S(t_5) = e^{-\left(\frac{t_5+t_2}{\lambda}\right)^\alpha} / e^{-\left(\frac{t_5}{\lambda}\right)^\alpha}$$

```
St7 <- exp(-(7000/la)^a)
St2 <- St7/St5
```

Example-3: Using 228 advanced lung cancer patients from the North Central Cancer Treatment Group (NCCTG). Variables used: time (survival days), status (1 = censored, 2 = dead), recoded as status2 (1 = event, 0 = censored).

Estimate the parameter and its 95% confidence interval based on an exponential distribution under Type-I censoring.

Solution: Using 228 advanced lung cancer patients from the North Central Cancer Treatment Group dataset, suppose the survival time follows an Exponential Distribution under Type-I censoring with censoring time $L = 365$ days.

Let the probability density function of exponential distribution for which survival time T is given by

$$f(t; \theta) = \frac{1}{\theta} e^{-\frac{t}{\theta}} ; t > 0$$

Now, for lung data the event indicator was recoded as,

$$\text{status2} = \begin{cases} 1 & ; \text{if death} \\ 0 & ; \text{if censored} \end{cases}$$

For Type-I censoring,

$$t_i = \min(T_i, L_i)$$

And

$$\delta_i = \begin{cases} 1 & ; \text{if } t_i = T_i \text{ or } T_i \leq L_i \\ 0 & ; \text{if } t_i = L_i \text{ or } T_i > L_i \end{cases}$$

```
library(survival)
lung$status2 <- ifelse(lung$status == 2, 1, 0)
names(lung)
table(lung$status2)
L <- 365
del <- sum(lung$status2)
t.i <- pmin(lung$time, L)
del.i <- as.integer(lung$time <= L & lung$status2 == 1)
```

Total sample size, $n = 228$.

Number of observed failures, $r = \sum \delta_i = 121$.

Total time on test, $T = \sum t_i = 55657$ days.

Under type-I censoring the MLE of the parameter θ is

$$\hat{\theta} = \frac{T}{r} = \frac{55657}{121} = 459.97 \approx 460$$


```
n <- nrow(lung)
r <- sum(del.i)
Tt <- sum(t.i)
cat("Events (death) = ", r, "| Censored = ", n-r, "\n")
theta.h <- Tt/r
```

Now, under large sample theory,

$$\hat{\theta} \sim N(\theta, [I(\theta)]^{-1})$$

Where,

$$I(\theta) = \frac{r}{\theta^2} \Rightarrow [I(\theta)]^{-1} = \text{Var}(\hat{\theta}) \approx \frac{\theta^2}{r}$$

So, the estimated standard error is

$$SE(\hat{\theta}) \approx \frac{\hat{\theta}}{\sqrt{r}} = \frac{460}{\sqrt{121}} = 41.82$$

Thus, the 95% confidence interval for θ is

$$\begin{aligned} & \hat{\theta} \pm 1.96 \times SE(\hat{\theta}) \\ &= 460 \pm 1.96(41.82) \\ &= 460 \pm 81.97 \\ &= (378.03, 541.97) \end{aligned}$$

is the 95% confidence interval for θ .

```
se.theta <- theta.h/sqrt(r)
ci.theta <- theta.h+c(-1,1)*1.96*se.theta
cat("Lower = ", ci.theta[1], "Upper = ", ci.theta[2], "\n")
```

Example-4: From the following data compute survival probabilities and estimate their standard errors. Draw a survival curve. From the survival curve estimate the probability that a participant survives 16 years and 21 years. Find the median survival year. Here number at risk at time $t = 0$ is 30.

T_i	0	1	2	3	5	6	9	10	11	12	13	14	17	18	19	21	23	24
D_i		2		1	2							1	2				2	
C_i			2			1	2	1	1	1	3	1	1	1	1	2	1	2

R Code

```
data<-data.frame(
  time=c(1,1,3,5,5,14,17,17,23,23,2,2,6,9,9,10,11,12,13,13,13,14,17,18,19,21,21,23,24,24),
  event=c(rep(1, 10), rep(0, 20)))
library(survival)
```

```
km<-survfit(Surv(time, event)~1, data=data)
summary(km)
plot(km,col="red",lwd=2,xlab="Time",ylab="Survival Probability",
      main="Kaplan-Meier Curves")
summary(km,times=c(16,21))
km
```

Example-5: 38 women were treated for infertility with laparoscopy and hydrotubation. All women were followed for up to 2-years to describe time-to conception (time-to-pregnancy). The event is conception, and women "survived" until they conceived. Estimate survival probabilities and plot the survival curve.

Conceived "Event" (month): 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 4, 4, 4, 6, 6, 9, 9, 9, 10, 13, 16
 Did not conceive "Censored" (month): 2.1, 3.1, 4, 7, 7, 8, 8, 9, 9, 9, 11, 24, 24

R Code

```
infert<-data.frame(time=c(1,1,1,1,1,1,2,2,2,2,2,3,3,3,4,4,4,6,6,9,9,9,
                          10,13,16,2.1,3.1,4,7,7,8,8,9,9,9,11,24,24),
                    event=c(rep(1, 25),rep(0, 13)))
library(survival)
km<-survfit(Surv(time, event)~1, data=infert)
summary(km)
plot(km, xlab = "Time", ylab = "Survival probability", main = "Kaplan Meier Plot", mark.time = TRUE)
abline(h=0.5, col="blue")
```

Example-6: Data for remission times in weeks for two groups of 42 leukemia patients are given below;

Group-1: Treatment group (21 patients)

6, 6, 6, 6⁺, 7, 9⁺, 10, 10⁺, 11⁺, 13, 16, 17⁺, 19⁺, 20⁺, 22, 23, 25⁺, 32⁺, 32⁺, 34⁺, 35⁺

Group-2: Placebo group (21 patients)

1, 1, 2, 2, 3, 4, 4, 5, 5, 8, 8, 8, 8, 11, 11, 12, 12, 15, 17, 22, 23

- Compute product limit estimates (K-M estimate) for the two groups and plot in a graph.
- Estimate the variance of survival function for both groups.
- Compute the 95% confidence interval for the survival time at time 16 for group-1.
- Compute the 95% confidence interval for the median remission time for group-1.
- For group-1, at 5% level of significance test the null hypothesis,

$$H_0 : S(t_{16}) = 0.9 \quad \text{vs} \quad H_1 : S(t_{16}) \neq 0.9$$

- Test the equality of the survival functions in the two groups.

R Code

```
mp<-data.frame(time=c(6,6,6,7,10,13,16,22,23,6,9,10,11,17,19,20,25,32,32,34,35),
                 event=c(rep(1, 9), rep(0, 12)))
placebo<-data.frame(time=c(1,1,2,2,3,4,4,5,5,8,8,8,8,11,11,12,12,15,17,22,23),
                     event=c(rep(1, 21), rep(0, 0)))
library(survival)
```

```

km1<-survfit(Surv(time, event)~1, data=mp)
km2<-survfit(Surv(time, event)~1, data=placebo)
summary(km1)
summary(km2)
plot(km1,col="red",lwd=2,xlab="Time",ylab="Survival Probability",
     main="Kaplan-Meier Curves",mark.time=TRUE,conf.int = FALSE)
lines(km2, col = "blue", lwd = 2,mark.time=TRUE,conf.int = FALSE)

summary(km1)$std.err^2
summary(km2)$std.err^2

summary(km1, times=16)

km1

S16<-summary(km1, times=16)$surv
SE16<-summary(km1, times=16)$std.err
Z<-(S16-0.9)/SE16
2*(1-pnorm(abs(Z)))

mp$group<-"mp"
placebo$group<-"placebo"
leuk<-rbind(mp, placebo)
library(survival)
survdiff(Surv(time,event)~group,data=leuk)

```

Example-7: In the dataset (schizophrenia.csv), you can find survival time (Time, time in days) for 280 cases of schizophrenia identified and followed for 5 years (1997-2001) in Ethiopia, together with the censoring indicator (Censor: 0 = censor, 1 = death), their marital status (Marital: 1 = single, 2 = married, 3 = Alone Again), gender (gender: 1 = Male, 2 = Female), schooling (in years), their age and their age at onset of schizophrenia.

- (a) Give the number of censored observations and events for all patients, but also by marital status and by gender.
- (b) Plot the Kaplan-Meier estimate of the survival curve for these patients.
- (c) Give the median survival time together with a 95% CI, and give also the survival time at 6 months, 18 months, and 36 months together with a 95% CI (specify which CI you provide).
- (d) Give the median survival time (with 95% CI) by marital status, and plot the Kaplan-Meier estimate of the survival curve by marital status
- (e) What can you conclude from these results? Give the median survival time (with 95% CI) by gender. Plot on the same graph the Kaplan-Meier survival function for patients with a certain gender. Shortly (1 sentence), interpret this plot.
- (f) Compare with Nelson-Aalen estimate at timepoints 90, 180, 270, 365 days.

Solution: Firstly, we have to read the data file and categories the required variables.

```
setwd("F:/Statistics semester wise book & sheet/8th semester/STA 4205/BioStat/")
schizo <- read.csv("Schizophrenia.csv", header=T, sep=",")
str(schizo)
schizo <- within(schizo, {
  sex <- factor(Gender, labels = c("Male", "Female"))
  censor <- factor(Censor, labels = c("Censor", "Death"))
  marital <- factor(Marital, labels = c("Single", "Married", "Alone again"))
  Time <- as.numeric(Time)
})
str(schizo)
```

- a.** The total number of censored observations is 117 and events (deaths) is 163.

```
table(schizo$censor)
sum(schizo$censor=="Censor")
```

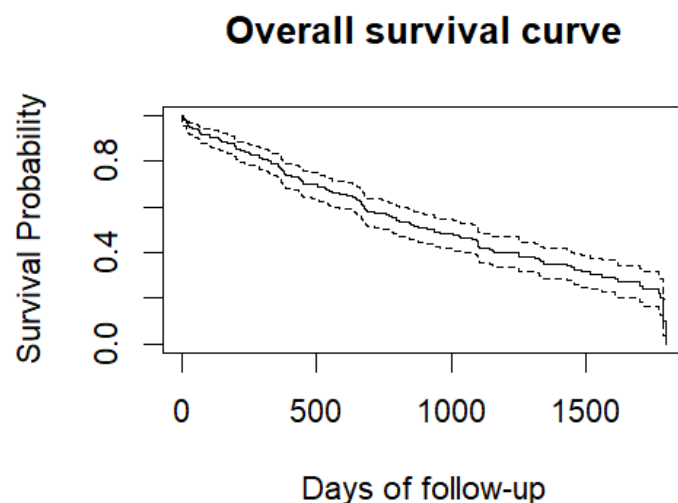
By marital status the number of censors is 50, 35 and 32 for Single, Married and Alone again.

```
table(schizo$censor, schizo$marital)
```

By gender the number of censors is 86 and 31 for Male and Female.

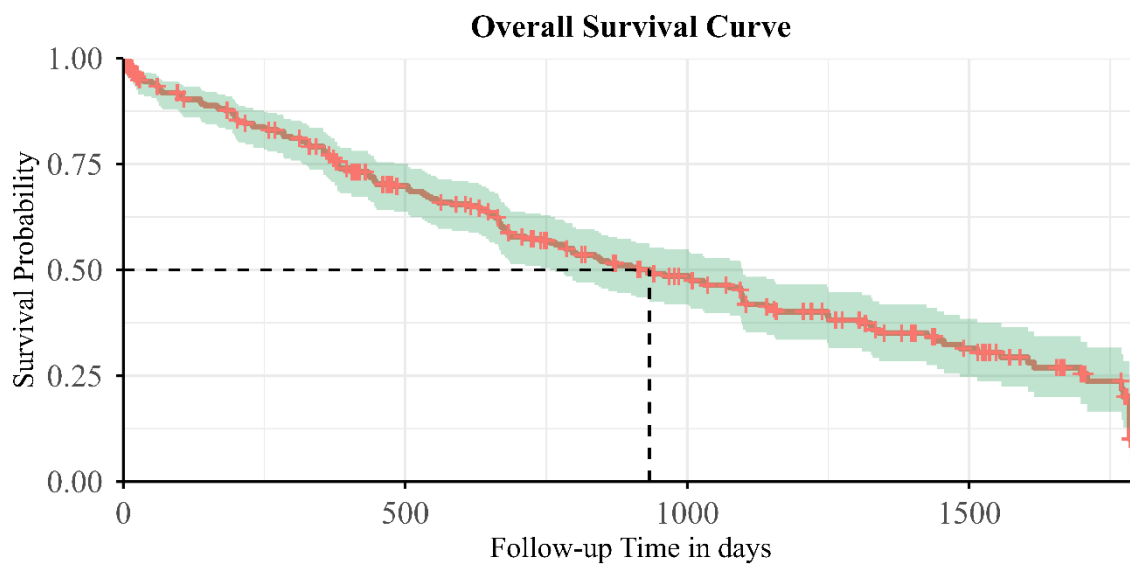
```
table(schizo$censor, schizo$sex)
```

- b.** Plotting Kaplan-Meier estimate of the survival curve for these patients.



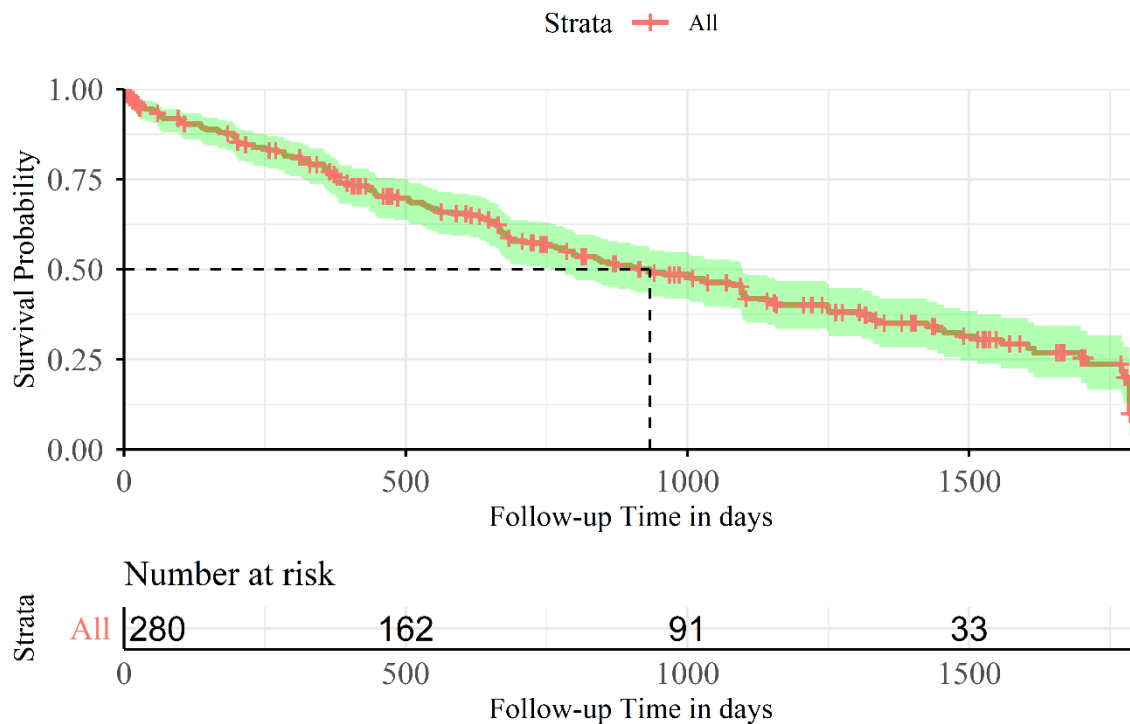
```
library(survival)
schizo_surv <- survfit(Surv(Time,Censor)~1, data = schizo, type = "kaplan-meier", conf.type
= "log-log")
print(schizo_surv) # Median survival
plot(schizo_surv, xlab = "Days of follow-up", ylab="Survival Probability", main= "Overall
survival curve")
```

Another way,



```
library(ggplot2)
library(survminer)
#install.packages("litedown", type="binary")
library(litedown)
ggsurvplot(schizo_surv,
  data = schizo,
  risk.table = F,
  pval = FALSE,
  conf.int = T,
  conf.int.fill = "#2ca25f",
  xlim = c(0,1800),
  title = "Overall Survival Curve",
  xlab = "Follow-up Time in days",
  ylab = "Survival Probability",
  surv.median.line = "hv",
  legend = "none",
  axes.offset = FALSE,    # Lines start at 0.0
  ggtheme = theme_minimal(base_family = "serif")+
  theme(
    plot.title = element_text(size = 12, face = "bold", hjust = 0.5),
    axis.text.x = element_text(size = 12),
    axis.text.y = element_text(size = 12),
    axis.line = element_line(color = "black")
  )
)
```

Another one,



```
library(ggplot2)
library(survminer)
ggsurvplot(
  schizo_surv,
  data = schizo,
  pval = F,
  conf.int = T,
  conf.int.fill = "green",
  xlim = c(0,1800),
  xlab = "Follow-up Time in days",
  ylab = "Survival Probability",
  surv.median.line = "hv",
  break.time.by = 500,
  axes.offset = FALSE,
  ggtheme = theme_minimal(base_family = "serif")+
  theme(
    axis.text.x = element_text(size = 12),
    axis.text.y = element_text(size = 12),
    axis.line = element_line(color = "black")
  ),
  risk.table = TRUE,
  risk.table.y.text.col = T,
  risk.table.y.text = T
)
```

c. The median survival time together with a 95% CI is

n	events	median	0.95LCL	0.95UCL
280	163	933	757	1099

The median survival time is 933 days.

```
print(schizo_surv)
```

The survival time at 6 months, 18 months and 36 months together with a 95% CI is

Time	n.risk	n.event	survival	std.err	lower 95% CI	upper 95% CI
180	231	32	0.881	0.0198	0.835	0.914
540	156	52	0.672	0.0295	0.611	0.727
1080	82	45	0.459	0.0333	0.392	0.522

The estimated survival probabilities at 6 months, 18 months and 36 months are 0.881, 0.672 and 0.459 respectively. This means that approximately 88.1%, 67.2% and 45.9% of patients survive beyond 6 months, 18 months and 36 months respectively.

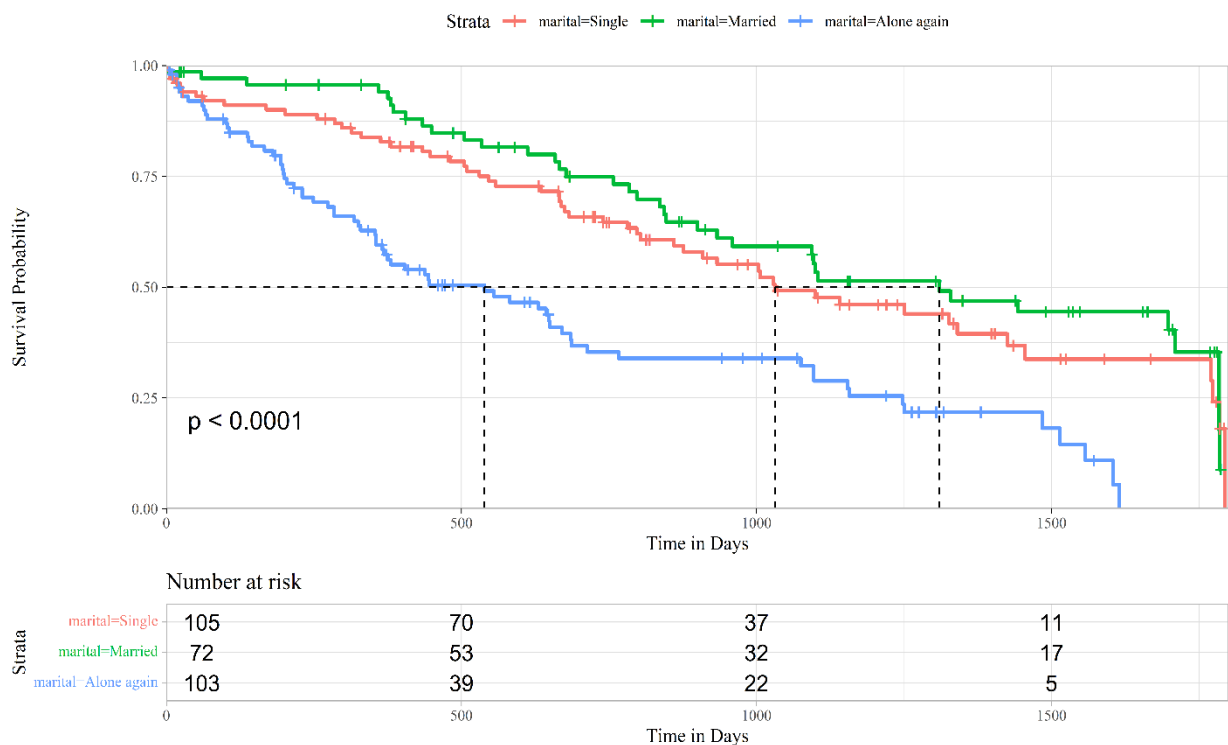
```
print(summary(schizo_surv, time=c(6*30, 18*30, 36*30)))
```

d. The median survival time with 95% CI by marital status.

	n	events	median	0.95LCL	0.95UCL
marital=Single	105	55	1031	803	1425
marital=Married	72	37	1310	900	1784
marital=Alone again	103	71	539	355	670

```
KM_schizo_mar<-survfit(Surv(Time,Censor) ~ marital, data=schizo, type="kaplan-meier",
conf.type="log-log")
print(KM_schizo_mar)
```

Kaplan-Meier estimate of the survival curve by marital status.



Since p-value < 0.0001, we conclude that highly significant differences in survival times among the three marital status groups. Patients who are 'Alone Again' have

the poorest survival (median 539 days), while married patients have the best survival (median 1310 days).

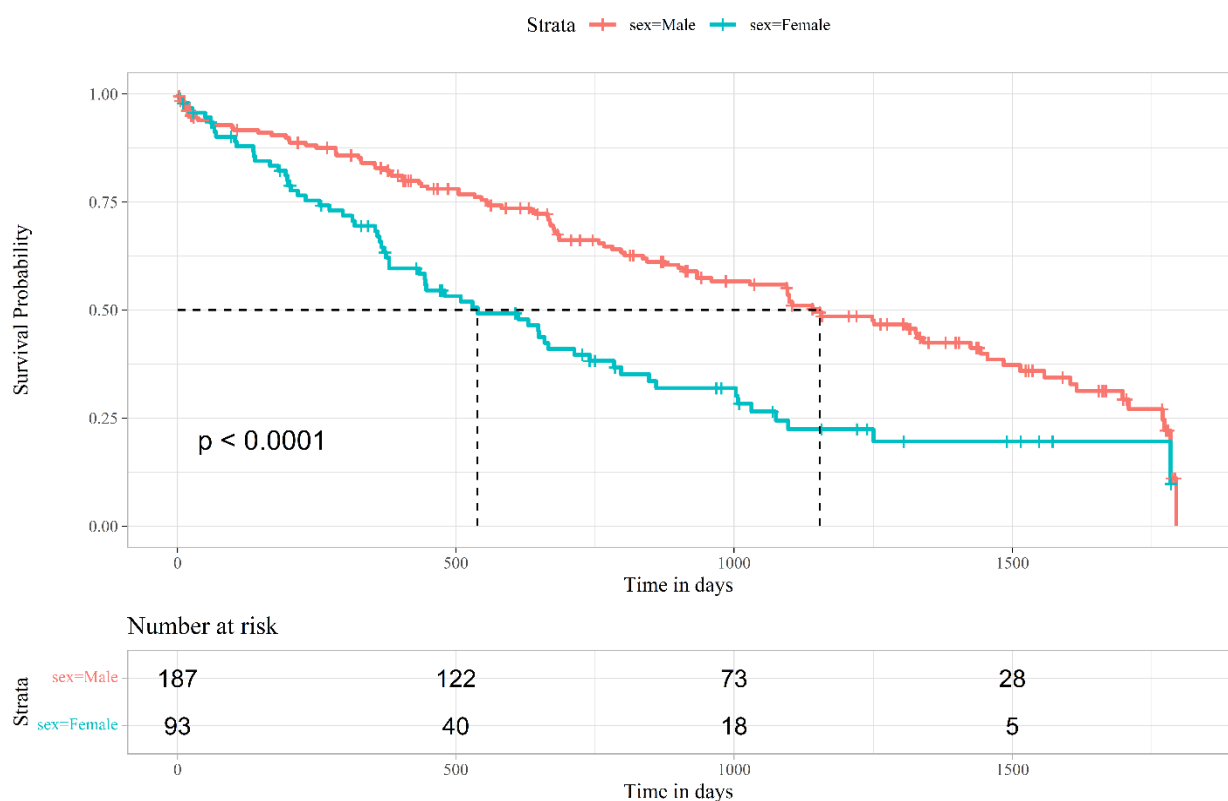
```
KM_schizo_mar <- survfit(Surv(Time, Censor) ~ marital, data=schizo, type="kaplan-meier",
  conf.type="log-log")
ggsurvplot(
  KM_schizo_mar,
  data = schizo,
  pval = TRUE,
  conf.int = F,
  xlim = c(0,1800),
  xlab = "Time in Days",
  ylab = "Survival Probability",
  break.time.by = 500,
  surv.median.line = "hv",
  ggtheme = theme_light(base_family = "serif"),
  risk.table = TRUE,
  risk.table.y.text.col = T,
  risk.table.y.text = T)
```

e. The median survival time with 95% CI by gender.

	n	events	median	0.95LCL	0.95UCL
sex=Male	187	101	1154	933	1425
sex=Female	93	62	539	380	740

Females have a substantially lower median survival time compared to males, indicating that females in this cohort have poorer survival outcomes.

Kaplan-Meier estimate of the survival curve by gender.



The Kaplan-Meier curves show that females have consistently lower survival probabilities than males throughout the follow-up period, with a log-rank p-value < 0.0001 confirming a statistically significant difference.

```
KM_schizo_gender <- survfit(Surv(Time,Censor) ~ sex, data=schizo, type="kaplan-meier",
conf.type="log-log")
print(KM_schizo_gender)
ggsurvplot(
  KM_schizo_gender,
  data = schizo,
  pval = TRUE,
  conf.int = F,
  xlim = c(0,1800),
  xlab = "Time in days",
  ylab = "Survival Probability",
  break.time.by = 500,
  surv.median.line = "hv",
  ggtheme = theme_light(base_family = "serif"),
  risk.table = TRUE,
  risk.table.y.text.col = T,
  risk.table.y.text = T)
```

- f.** The comparison between the Kaplan-Meier Estimator and Nelson-Aalen Estimator estimates at timepoints 90, 180, 270 and 365 days.

Time	KM S	KM lower	KM upper	NA S	NA lower	NA upper
90	0.9188	0.8791	0.9457	0.9188	0.8793	0.9458
180	0.8809	0.8354	0.9141	0.8809	0.8357	0.9142
270	0.8310	0.7799	0.8708	0.8310	0.7803	0.8710
365	0.7723	0.7161	0.8180	0.7723	0.7166	0.8184

In general, NA is greater or equal. This result indicates that both methods provide very similar estimates of the survival experience of the patients.

```
na_fit <- survfit(Surv(Time, Censor) ~ 1, data = schizo, type = "fh", conf.type = "log-log")
schizo_surv <- survfit(Surv(Time, Censor) ~ 1, data = schizo, type = "kaplan-meier",
conf.type = "log-log")
na_summary <- summary(na_fit, times = c(90, 180, 270, 365))
km_summary <- summary(schizo_surv, times = c(90, 180, 270, 365))
comparison <- data.frame(
  Time = na_summary$time,
  KM_S = round(na_summary$surv, 4),
  KM_lower = round(km_summary$lower, 4),
  KM_upper = round(km_summary$upper, 4),
  NA_S = round(na_summary$surv, 4),
  NA_lower = round(na_summary$lower, 4),
  NA_upper = round(na_summary$upper, 4)
)
print(comparison)
```

Question-2023: You can find the dataset ART.csv from the [STA42061.zip; Password: StA42061.] on your Desktop. Please provide all codes, outputs, and interpretations in your exam paper. Also, roughly draw the plot by hand on the exam paper. This dataset contains the following information:

Variable Name	Description
Patid	Patient Identification number (1-100)
Time	Time (in months) to censoring or death
Censor	Censoring status (0=censored 1 death)
Gender	Gender (1=male 2=female)
Weight	The weight (in kg)
Function	The functional status (0=bedridden, 1=ambulatory, 2=working)
Clinstage	The WHO clinical stage
CD4	The CD4 count
ART	The ART (1=D4T(30)-3TC-NVP, 2=D4T(40)-3TC-NVP, 3=AZT-3TC-NVP)

- IMPORT data ART.csv using the name ART (write down the code in your exam paper).
- Randomly select 82 patients from the ART dataset using your ID number as the seed number. Use this code: `set.seed(ID Number)`
`ART2 <- ART [sample(nrow(ART), size=82, replace = FALSE).]`

Use the randomly selected dataset of Antiretroviral Therapy (ART) for HIV treatment of 82 patients. Answer the following questions:

- Provide the number and proportion of censored observations and events for all patients and by functional status and gender.
- Plot the Kaplan-Meier estimated survival curve for all patients with a 95% confidence interval and interpret this plot. Specify the survival at 15 years, 25 years, and 35 years (with 95% CIs). What can you say about the median survival time?
- Plot the Kaplan-Meier estimated survival curve by the ART therapy given to HIV patients and provide the ART therapy's median survival time (with 95% CI). What can you conclude from these results?

Example-8: The dataset Hypertension. The data are derived from a hypertension screening study. Of the 727 participants of this study who were examined, the following data are included in the file Hypertension_Lab.csv:

Variable's Name	Label	Category
ID	Patient's number	ID Number
bmi	Patient's body mass index	Continuous scale
sleep	Patient's sleeping hours	Continuous scale
hypert	Patient's hypertension status	0=No, 1=Yes
Gender	Sex of patient's	0=Male, 1=Female

Using R or RStudio to answer the following question;

- Are there any missing observations? If yes, how many of them?
- Recode/categorize BMI into bmi_cat (Normal: $\text{bmi} < 25$; Overweight: $\text{bmi} < 30$; Obese: $\text{bmi} \geq 30$).

- (c) Factorize all required variables in question (g).
- (d) Find the proportion table of gender.
- (e) What is the risk of hypertension for male? Give also the 95% CI for the risk.
- (f) Is there any association between the hypertension and bmi_cat? Which category has the highest probability of hypertension? Is the variable bmi_cat significant?
- (g) Fit a logistic regression model to predict hypertension, using bmi, sleep, gender as main effects.
- (h) Find the odds ratios and interpret the significant factors.
- (i) Calculate 95% confidence of all factors.
- (j) Find the deviance and AIC.
- (k) Calculate the sensitivity, recall, specificity, PPV, precision, NPV, F1 score and accuracy level of the test and interpret.
- (l) Calculate multicollinearity by variance inflation factor and interpret.
- (m) Test the multicollinearity using Hosmer-Lemeshow goodness of fit.
- (n) Find McFadden's pseudo- R^2 .
- (o) Draw the ROC curve.
- (p) Calculate the area under curve value and interpret it.
- (q) Calculate average marginal effect and interpret.

Solution: First of all, we have to read dataset.

```
setwd("F:/Statistics semester wise book & sheet/8th semester/STA 4205/BioStat/")
HT <- read.csv("Hypertension_Lab.csv", header = T, sep = ",")
```

- a.** Finding missing value in the dataset;

```
sum(is.na(HT))
[1] 0
```

There is no missing value.

- b.** Recode/categorize BMI into bmi_cat;

```
HT$bmi_cat <- ifelse(HT$bmi < 25, "Normal",
  ifelse(HT$bmi < 30, "Overweight", "Obese"))
```

- c.** Factorize all required variables;

```
HT <- within(HT, {
  bmi_cat <- factor(bmi_cat, levels = c("Normal", "Overweight", "Obese"))
  sex <- factor(Gender, labels = c("Male", "Female"))
  ht <- factor(hypert, labels = c("No", "Yes"))
})
```

- d.** The proportion table of gender;

```
prop.table(table(HT$sex))
```

- e.** Table of gender with hypertension;

```
table(HT$ht, HT$sex)
```

	Male	Female
No	188	297
Yes	106	136

The risk of hypertension for male,

$$\hat{p} = \frac{106}{188 + 106} = 0.3605$$

Therefore, 95% CI for the risk,

$$\hat{p} \pm 1.96 \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}} = 0.3605 \pm 1.96 \times 0.0279 = (0.3057, 0.4152)$$

f. Test the association between the hypertension and bmi_cat;

```
t<-table(HT$bmi_cat, HT$ht)
t
```

	No	Yes
Normal	215	58
Overweight	171	66
Obese	99	118

```
chisq.test(t)
X-squared = 64.45, df = 2, p-value = 1.011e-14
```

Since p-value is very small, there is a association between the hypertension and bmi_cat and the variable bmi_cat is significant.

```
prop.table(t,2)
```

	No	Yes
Normal	0.4432990	0.2396694
Overweight	0.3525773	0.2727273
Obese	0.2041237	0.4876033

Obese bmi category has the highest probability of hypertension.

g. Fitting logistic model to predict hypertension;

```
model <- glm(ht~bmi+sleep+sex, data=HT, family=binomial(link= "logit"))
```

h. The odds ratios;

```
library(broom)
tidy(model,exponentiate=T)
```

term	estimate	std.error	statistic	p.value
(Intercept)	0.0330	0.458	-7.44	9.95e-14
bmi	1.11	0.0138	7.48	7.72e-14
sleep	0.986	0.0236	-0.612	5.40e-1
sexFemale	0.824	0.167	-1.16	2.47e-1

Holding sleep duration and gender constant, each one-unit increase in BMI increases the odds of hypertension by approximately 11%.

After adjusting for BMI and gender, each additional hour of sleep decreases the odds of hypertension by about 1.4%.

After adjusting for BMI and sleep, females have approximately 17.6% lower odds of hypertension than males.

i. 95% confidence of all factors;

library(broom)						
tidy(model,exponentiate=T)						
term	estimate	std.error	statistic	p.value	conf.low	conf.high
(Intercept)	0.0330	0.458	-7.44	9.95e-14	0.0135	0.0854
bmi	1.11	0.0138	7.48	7.72e-14	1.08	1.14
sleep	0.986	0.0236	-0.612	5.40e-1	0.913	1.02
sexFemale	0.824	0.167	-1.16	2.47e-1	0.594	1.14

j. The model deviance and AIC;

```
cat("\nNull deviance :", model$null.deviance)
cat("\nResidual deviance:", model$deviance)
cat("\nAIC :", AIC(model))
```

k. The sensitivity, recall, specificity, PPV, precision, NPV, F1 score and accuracy level of the test;

```
predicted_prob <- predict(model, type="response")
predicted_class<-ifelse(predicted_prob > 0.5, 1, 0)
classification_table<- table(Actual = HT$ht, Predicted = predicted_class)
classification_table
TN <- classification_table[1, 1]
FP <- classification_table[1, 2]
FN <- classification_table[2, 1]
TP <- classification_table[2, 2]
sen<-TP/(TP+FN)
rec<-TP/(TP+FN)
spc<-TN/(TN+FP)
ppv<-TP/(TP+FP)
prec<- TP/(TP + FP)
npv<-TN/(TN+FN)
f1<- 2*(TP/(TP+FP))*(TP/(TP+FN))/((TP/(TP+FP))+(TP/(TP+FN)))
acc<-(TP+TN)/(TP+TN+FP+FN)
cat("\n Sensitivity =",sen,"\n", "Recall =",rec, "\n", "Specificity =",spc, "\n", "PPV =",ppv,"\n", "Precision =",prec, "\n", "NPV =",npv,"\n", "F1 score =",f1, "\n", "Accuracy =",acc,"\n")

Sensitivity = 0.2479339
Recall = 0.2479339
Specificity = 0.9319588
PPV = 0.6451613
Precision = 0.6451613
NPV = 0.7129338
F1 score = 0.358209
Accuracy = 0.7042641
```

The model correctly identifies only 24.8% of patients with hypertension.

The model correctly identifies 93.2% of non-hypertensive patients.

Among individuals predicted to have hypertension, 64.5% actually have hypertension. Among individuals predicted not to have hypertension, 71.3% are actually non-hypertensive.

The F1 score is relatively low, indicating that the model has poor balance between precision and sensitivity.

Overall, the model correctly classifies approximately 70.4% of all participants.

l. Multicollinearity by variance inflation factor;

```
library(car)
vif(model)
      bmi      sleep      sex
1.004756 1.005442 1.000874
```

All VIF values are approximately 1 (less than 5), indicating no multicollinearity among the explanatory variables.

m. Test the multicollinearity using Hosmer-Lemeshow;

```
library(ResourceSelection)
hoslem.test(x = as.numeric(HT$hypert), y = fitted(model), g = 10)
```

n. McFadden's pseudo-R²;

```
1-(model$deviance/model$null.deviance)
0.0744
```

o. For ROC curve;

```
library(pROC)
roc_obj <- roc(response = HT$ht, predictor = fitted(model), levels = c("No",
"yes"))
plot(roc_obj)
```

p. The area under curve value;

```
auc(roc_obj)
Area under the curve: 0.6851
```

Approximately 68.5% probability that the model assigns a higher predicted probability of hypertension to a randomly selected hypertensive individual than to a randomly selected non-hypertensive individual.

q. Average marginal effect;

```
library(marginaleffects)
ame <- avg_slopes(model)
ame
```

Term	Contrast	Estimate	Std. Error	z	Pr(> z)	S
bmi	dY/dX	0.0208	0.00238	8.742	<0.001	58.6
sex	Female - Male	-0.0391	0.03392	-1.153	0.249	2.0
sleep	dY/dX	-0.0029	0.00474	-0.612	0.540	0.9

Holding all other variables constant, each one-unit increase in BMI increases the probability of hypertension by approximately 2.08 percentage points on average. After controlling for BMI and sleep, females have an average 3.91 percentage-point lower probability of hypertension than males.

Holding other variables constant, an additional hour of sleep decreases the probability of hypertension by approximately 0.29 percentage points on average.

Midterm Question-2024: The dataset (biostat.csv) contains 169 records of bone densitometries (measurement of bone mineral density). The following variables were collected:

Variable's Name	Label	Category
id	Patient's number	ID Number
age	Patient's age	Continuous in years
sex	Patient's gender	1=M, 0=F
fracture	Hip fracture	1=fracture, 0=no fracture
weight kg	Weight measured in Kg	Continuous in kg
height cm	Height measured in cm	Continuous in cm
medication	Medication	0=No medication, 1=Anticonvulsant, 2=Glucocorticoids
waiting_time	Time the patient had to wait for the densitometry (in minutes)	Continuous in minutes
bmd	Bone mineral density measure in the hip	Continuous scle

Using R or RStudio to answer the following question;

- Fit a logistic regression model to predict fracture, using AGE, SEX, BMI, Medication, Waiting Time and BMD as main effects. Interpret the significant factors.
- Calculate the sensitivity, specificity, PPV, NPV and accuracy level of the test. What is the value of AUC? Interpret all the measures.

Solution: First of all, we have to read dataset and modify our required variables to fit a logistic regression model to predict fracture.

Since we have to use AGE, SEX, BMI, Medication, Waiting Time and BMD;

```
setwd("F:/Statistics semester wise book & sheet/8th semester/STA 4205/BioStat/STA L4206 Midterm/")
data<-read.csv("biostat.csv", header = T)
data<-within(data,{
  gen<-factor(sex, levels=c("F","M"))
  frac<-factor(fracture, levels=c("no fracture","fracture"))
})
```



```

medi<-factor(medication, levels=c("No medication", "Anticonvulsant", "Glucocorticoids"))
height_m<-height_cm/100
bmi<-weight_kg/(height_m)^2
})

```

- a.** Fitting regression model to predict fracture, using AGE, SEX, BMI, Medication, Waiting Time and BMD as main effects.

```

model<-glm(frac ~ age + gen + bmi + medi + waiting_time + bmd , data=data, family =
binomial (link = "logit"))
library(broom)
tidy(model,conf.int=T,exponentiate=T)
summary(model)

```

term	estimate	std.error	statistic	p.value
(Intercept)	5.93e+4	2.98	3.69	0.000224
age	1.02e+0	0.0218	0.742	0.458
genM	2.74e+0	0.536	1.88	0.0605
bmi	9.62e-1	0.0603	-0.635	0.526
mediAnticonvulsant	5.40e-1	0.957	-0.644	0.520
mediGlucocorticoids	1.60e-1	0.867	-2.11	0.0348
waiting_time	1.01e+0	0.0148	0.467	0.641
bmd	4.27e-8	3.18	-5.34	0.0000000923

Hence, Male at 10%, medication Glucocorticoids at 1% and bmd (Bone mineral density) at 5% are significant.

Male patients have approximately 2.74 times higher odds of experiencing a hip fracture than female patients holding all other variables constant.

Glucocorticoids have 0.16 times the odds (or 84% lower) of hip fracture compared with patients receiving no medication after adjusting other variables.

One-unit increase in bone mineral density (bmd) is associated with an odd of hip fracture that is 4.27×10^{-8} times as large, indicating a very large reduction in the odds of fracture holding all other variables constant.

- b.** To calculate the sensitivity, specificity, PPV, NPV and accuracy level of the test;

```

pred<-predict(model,type="response")
pred<-ifelse(pred>0.5,1,0)
pred<-factor(pred,levels=c(0,1),labels=c("No","Yes"))
cls<-table(Actual=data$frac,Predict=pred)
cls
TN<-cls[1,1]
FP<-cls[1,2]
FN<-cls[2,1]
TP<-cls[2,2]
sen<-TP/(TP+FN)
spc<-TN/(TN+FP)
ppv<-TP/(TP+FP)

```



```
npv<-TN/(TN+FN)
acc<-(TP+TN)/(TP+TN+FP+FN)
cat("\n Sensitivity =",sen,"\n", "Specificity =",spc, "\n","PPV =",ppv,"\n", "NPV
=",npv,"\n", "Accuracy =",acc,"\n")
```

We get, Sensitivity = 0.78, Specificity = 0.9159664, PPV = 0.7959184, NPV = 0.9083333, Accuracy = 0.8757396

The model correctly identifies 78% of patients who actually have a hip fracture.

The model correctly identifies 91% of patients who do not have a hip fracture.

Among patients predicted to have a hip fracture, 79.6% actually have a fracture.

Among patients predicted not have fracture, 90.8% actually do not have a fracture.

The model correctly classifies 87% of all patients.

To find area under curve (AUC);

```
library(pROC)
pred<-predict(model, type="response")
roc.obj<-roc(data$frac,pred)
auc(roc.obj)
```

Area under the curve: 0.9269

Approximately 93% probability that the model assigns a higher predicted probability of fracture to a randomly selected patient with fracture than without fracture.

Question-2023: The dataset [STA4206L.zip; Password: StA42061.]. The data are derived from a hypertension screening study of the elderly people. Of the 1596 participants of this study who were examined in 1993 as well as in 1995, the following data are included in the file HYPERT.csv:

Variable	Description	Codes/Values
Idnr	Identification number	1-1596
sex	Gender	0=male 1=female
age	Age in 1993	Years
height	Height in 1993	cm
weight	Weight in 1993	kg
smoke93	Smoking in 1993	0=no 1=yes
sbp93	Systolic Blood Pressure in 1993	mmHg
sbp95	Systolic Blood Pressure in 1995	mmHg
ISII	Isolated Systolic Hypertension in 1993	0=no 1=yes

You can find the dataset HYPERT.csv from the STA4206L.zip folder. You can solve the questions by using R or SPSS. Write down all codes and relevant outputs in your answer script.

R Instruction

- IMPORT data HYPERT.csv using the name HYPERT (write down the code in your exam paper). Randomly select 650 cases from the HYPERT dataset using your ID number as the seed number (set.seed(ID Number) (Use this code:
HYPERT2<- HYPERT [sample(nrow(HYPERT), size 650, replace FALSE)].J
- Export the data into SPSS file (sav) using haven package
[write_sav(HYPERT2,"HYPERT.sav")].

Work on the randomly selected data file and answer the following questions.

- (a) Are there any missing observations? If yes, how many of them?
- (b) Calculate BMI in the IYPERT2 dataset and then recode/categorize BMI into BMICLASS (1: $\text{BMI} < 20$; 2: $20 \leq \text{BMI} < 25$; 3: $25 \leq \text{BMI} < 30$; and 4: $\text{BMI} \geq 30$).
- (c) What is the risk of Isolated Systolic Hypertension in 1993 for women? Give also the 95% confidence interval for this risk (calculate by hand and report it in your exam script).
- (d) Is there any association between the Isolated Systolic Hypertension in 1993 and BMICLASS? Which BMICLASS has the highest probability of ISII? Is the variable BMICLASS significant?
- (e) Run a logistic regression model using the response variable ISII and the explanatory variables AGE, SEX, SMOKE93, and BMICLASS. Explain the output in terms of odds ratio.

Operation Research and Biostatistics Lab

Mohammad Sobrul
Department of Statistics
Jagannath University, Dhaka-1100

Operations Research (Lab), STAL 4206

Problem # 01

A garment manufacturer has a production line making two styles of shirts. Style I needs 100 g of cotton thread, 200 g of Dacron thread and 200 g of linen thread. Corresponding requirements of style II are 100g, 300g and 200g. The net contributions are 100 Taka for style I and 120 Taka for style II. The available inventory of cotton thread, Dacron thread and linen thread are, respectively, 34 kg, 36 kg and 32 kg. The manufacturer wants to determine the number of each style to be produced with the given inventory. Formulate the LPP model and solve it by graphical method using R code.

Solution:

Step 1: Define Decision Variables

Let:

x_1 = number of Style I shirts to produce

x_2 = number of Style II shirts to produce

Step 2: Objective Function

Profit (in Taka):

$$\text{Maximize } Z = 100x_1 + 120x_2$$

Step 3: Constraints

Convert kg to gram (g):

34 kg = 34,000 g, 36 kg = 36,000 g, 32 kg = 32,000 g

Thread constraints:

Resource	Style I (g)	Style II (g)	Available (g)
Cotton Thread	100	100	34,000
Dacron Thread	200	300	36,000
Linen Thread	200	200	32,000

So the constraints are:

$$100x_1 + 100x_2 \leq 34000 \text{ (Cotton)}$$

$$200x_1 + 300x_2 \leq 36000 \text{ (Dacron)}$$

$$200x_1 + 200x_2 \leq 32000 \text{ (Linen)}$$

$$x_1, x_2 \geq 0$$

R code:

```
library(ggplot2)
#install.packages("ggplot2")
# Define functions for constraint lines
x <- seq(0, 400, 1)

cotton <- (34000 - 100*x) / 100
dacron <- (36000 - 200*x) / 300
linen <- (32000 - 200*x) / 200

# Feasible region
df <- data.frame(x = x,
                 cotton = pmax(0, cotton),
                 dacron = pmax(0, dacron),
                 linen = pmax(0, linen))

# Plot constraints
ggplot(df, aes(x)) +
  geom_line(aes(y = cotton, color = "Cotton Constraint")) +
  geom_line(aes(y = dacron, color = "Dacron Constraint")) +
  geom_line(aes(y = linen, color = "Linen Constraint")) +
  xlim(0, 200) + ylim(0, 200) +
  labs(title = "Feasible Region for Shirt Production",
       x = "Style I (x1)",
       y = "Style II (x2)") +
  scale_color_manual(values = c("blue", "green", "red")) +
  theme_minimal()

# Find corner points of feasible region
# Manually solve equations at intersection points

pts <- data.frame(
  x1 = c(0, 0, 160, 100, 40),
  x2 = c(160, 120, 0, 40, 120)
)
pts$Z <- 100 * pts$x1 + 120 * pts$x2

opt_idx <- which.max(pts$Z)
opt_point <- pts[opt_idx, ]

cat("Optimal solution:\n")
```

```
cat("Style I (x1) =", opt_point$x1, "\n")
cat("Style II (x2) =", opt_point$x2, "\n")
cat("Maximum Profit (Z) =", opt_point$Z, "Taka\n")
```

Problem # 02

An animal feed company must produce 200 kg of a mixture consisting of ingredients A and B daily. A costs Rs. 3 per kg and B costs Rs. 8 per kg. Not more than 80 kg of A can be used and at least 60 kg of B must be used.

The company wants to know how much of each ingredient should be used to minimize cost. Formulate the LPP.

Solution:

Problem Summary

- Let:
 - x_1 = kg of ingredient A
 - x_2 = kg of ingredient B
- Objective:

Minimize Cost: $Z = 3x_1 + 8x_2$
- Subject to constraints:
 1. $x_1 + x_2 = 200$ (total 200 kg required)
 2. $x_1 \leq 80$ (not more than 80 kg of A)
 3. $x_2 \geq 60$ (at least 60 kg of B)
 4. $x_1, x_2 \geq 0$ (non-negativity)

R code:

```
# Install and load lpSolve if needed

if (!require(lpSolve)) install.packages("lpSolve", dependencies = TRUE)

library(lpSolve)

# Objective function coefficients (minimize 3x1 + 8x2)
objective <- c(3, 8)

# Constraint matrix
```

```

# Each row corresponds to a constraint
# Constraints:
# x1 + x2 = 200
# x1 <= 80
# x2 >= 60
constraints <- matrix(c(
  1, 1,    # x1 + x2 = 200
  1, 0,    # x1 <= 80
  0, 1     # x2 >= 60
), nrow = 3, byrow = TRUE)

# Direction of each constraint
directions <- c("=", "<=", ">=")

# Right-hand side of constraints
rhs <- c(200, 80, 60)

# Solve the LP using simplex method (default)
solution <- lp(direction = "min",
  objective.in = objective,
  const.mat = constraints,
  const.dir = directions,
  const.rhs = rhs)

# Print optimal values of decision variables
cat("Optimal values (x1, x2):", solution$solution, "\n")

# Print minimum cost
cat("Minimum Cost (Z):", solution$objval, "\n")

```

Graphically solved by R

```

# Load required package
library(ggplot2)

# Create a sequence for x1
x1 <- seq(0, 200, 1)

# Constraint 1: x1 + x2 = 200 => x2 = 200 - x1
x2_eq <- 200 - x1

# Constraint 3: x2 >= 60 (constant line)
x2_min <- rep(60, length(x1))

# Constraint 2: x1 <= 80 (vertical line at x1 = 80)

# Find feasible region
feasible_df <- data.frame()
for (i in 0:80) {
  x2_val <- 200 - i
  if (x2_val >= 60) {
    feasible_df <- rbind(feasible_df, data.frame(x1 = i, x2 = x2_val))
  }
}

# Objective function line (Z = 3x1 + 8x2)
# Let's plot for Z = 1600, 1700, 1800 for visual clarity
z1 <- (1600 - 3 * x1) / 8
z2 <- (1700 - 3 * x1) / 8
z3 <- (1800 - 3 * x1) / 8

# Plotting

```

```

ggplot() +
  geom_line(aes(x = x1, y = x2_eq), color = "blue", size = 1, linetype =
"solid") +
  geom_line(aes(x = x1, y = z1), color = "red", linetype = "dashed") +
  geom_line(aes(x = x1, y = z2), color = "red", linetype = "dashed") +
  geom_line(aes(x = x1, y = z3), color = "red", linetype = "dashed") +
  geom_hline(yintercept = 60, color = "darkgreen", linetype = "dotted", size
= 1) +
  geom_vline(xintercept = 80, color = "purple", linetype = "dotted", size =
1) +
  geom_point(data = feasible_df, aes(x = x1, y = x2), color = "orange") +
  geom_point(aes(x = 80, y = 120), color = "black", size = 3) + # Optimal
point
  annotate("text", x = 82, y = 122, label = "Optimal (80, 120)", hjust = 0) +
  labs(title = "Linear Programming: Graphical Solution",
       x = "x1 (kg of Ingredient A)",
       y = "x2 (kg of Ingredient B)") +
  theme_minimal()

```

Problem # 03

Solve the following problem using R code:

A factory has engaged in manufacturing three products A, B and C which involve lathe work, grinding and assembling. The cutting, grinding and assembling times required for one unit of A are 2, 1 and 1 hour respectively. Similarly they were 3, 1 and 3 hours for one unit of B and 1, 3 and 1 hours for one unit of C. The profits on A, B and C are TK.2, TK.2 and TK. 4 per unit respectively. Assuming that there are available 300 hours of lathe time, 300 hours of grinder time and 240 hours of assembly time. How many units of each product should be produced to maximize profit? Formulate the problem mathematically.

Solution:

Mathematical Formulation

Let:

- x_1 = units of product A
- x_2 = units of product B
- x_3 = units of product C

Objective Function (Profit):

Maximize:

$$Z = 2x_1 + 2x_2 + 4x_3$$

Subject to Time Constraints:

- **Lathe time:** $2x_1 + 3x_2 + 1x_3 \leq 300$
- **Grinding time:** $1x_1 + 1x_2 + 3x_3 \leq 300$
- **Assembly time:** $1x_1 + 3x_2 + 1x_3 \leq 240$
- **Non-negativity:** $x_1, x_2, x_3 \geq 0$

R code to solve the problem:

```
# Install and load lpSolve if not already installed
if (!require(lpSolve)) install.packages("lpSolve", dependencies = TRUE)
library(lpSolve)

# Objective function coefficients (profit per unit)
objective <- c(2, 2, 4) # x1 = A, x2 = B, x3 = C

# Constraint matrix
# Rows = constraints (lathe, grinding, assembly)
# Columns = variables (x1, x2, x3)
constraints <- matrix(c(2, 3, 1, # Lathe
                        1, 1, 3, # Grinding
                        1, 3, 1), # Assembly
                      byrow = TRUE, nrow = 3)

# RHS of the constraints
rhs <- c(300, 300, 240)

# Direction of constraints
directions <- c("<=", "<=", "<=")

# Solve the problem
solution <- lp(direction = "max",
               objective.in = objective,
               const.mat = constraints,
               const.dir = directions,
               const.rhs = rhs)

# Print results
if (solution$status == 0) {
  cat("Optimal solution found:\n")
  cat("Produce", solution$solution[1], "units of Product A\n")
  cat("Produce", solution$solution[2], "units of Product B\n")
  cat("Produce", solution$solution[3], "units of Product C\n")
  cat("Maximum Profit: TK.", solution$objval, "\n")
} else {
  cat("No optimal solution found. Status code:", solution$status, "\n")
}
```


#Alternative way to display the results

```
# Print optimal values of decision variables
cat("Optimal values (x1, x2, x3):", solution$solution, "\n")

# Print minimum cost
cat("Maximum value Profit (Z):", solution$objval, "\n")
```

Problem # 04

Solve the following LPP using R code by Simplex method

Minimize:

$$Z = x_1 + x_2 - 4x_3$$

Subject to:

$$x_1 + x_2 + 2x_3 \leq 8$$

$$x_1 + x_2 - x_3 \leq 2$$

$$-x_1 + x_2 + x_3 \leq 4$$

$$x_1, x_2, x_3 \geq 0$$

Solution:

```
# Install lpSolve if not already installed
if (!require(lpSolve)) install.packages("lpSolve", dependencies=TRUE)
library(lpSolve)
```

```
# Objective function coefficients
objective <- c(1, 1, -4) # Coefficients for x1, x2, x3
```

```
# Constraint coefficients
constraints <- matrix(c(1, 1, 2,
                        1, 1, -1,
                        -1, 1, 1),
                      nrow=3, byrow=TRUE)
```

```
# Right-hand side of constraints
rhs <- c(8, 2, 4)
```

```
# Direction of constraints
direction <- c("<=", "<=", "<=")
```

```
# Solve the problem (minimization)
solution <- lp(direction = "min",
               objective.in = objective,
               const.mat = constraints,
               const.dir = direction,
```

```

const.rhs = rhs)

# Display results
if (solution$status == 0) {
  cat("Optimal solution found:\n")
  cat("x1 =", solution$solution[1], "\n")
  cat("x2 =", solution$solution[2], "\n")
  cat("x3 =", solution$solution[3], "\n")
  cat("Minimum Z =", solution$objval, "\n")
} else {
  cat("No optimal solution found. Status code:", solution$status, "\n")
}

#Alternative way to display the results
cat("Optimal values (x1, x2, x3):", solution$solution, "\n")
cat("Minimum value (Z):", solution$objval, "\n")

```

Problem # 05

Solve the following LPP by dual simplex method using R code

$$\begin{aligned}
 &\text{Minimize} && Z = 5x_1 + 6x_2 \\
 &\text{Subject to,} && x_1 + x_2 \geq 2 \\
 &&& 4x_1 + x_2 \geq 4 \\
 &&& x_1, x_2 \geq 0
 \end{aligned}$$

Solution:

```

# Load lpSolve package
#Install.packages("lpSolve")

library(lpSolve)

# Objective function coefficients
objective <- c(5, 6)

# Constraints matrix (converted to <= form)
const_mat <- matrix(c(-1, -1,
                      -4, -1),
                    nrow = 2, byrow = TRUE)

# Right-hand side of constraints
const_rhs <- c(-2, -4)

# Directions of constraints (all <= after conversion)
const_dir <- c("<=", "<=")

# Solve the LP using lp function
solution <- lp(direction = "min",

```

```

        objective.in = objective,
        const.mat = const_mat,
        const.dir = const_dir,
        const.rhs = const_rhs,
        compute.sens = TRUE)

# Print results
if (solution$status == 0) {
  cat("Optimal solution found:\n")
  cat("x1 =", solution$solution[1], "\n")
  cat("x2 =", solution$solution[2], "\n")
  cat("Minimum z =", solution$objval, "\n")
} else {
  cat("No optimal solution found.\n")
}

#Alternative way to display the results
cat("Optimal values (x1, x2):", solution$solution, "\n")
cat("Minimum value (Z):", solution$objval, "\n")

```

Problem # 06

Solve the following LPP by Big-M method using R code

$$\begin{array}{ll}
 \text{Minimize} & Z = 3x_1 + 2x_2 \\
 \text{Subject to,} & 2x_1 + x_2 \geq 10 \\
 & -3x_1 + 2x_2 \leq 6 \\
 & x_1 + x_2 \geq 6 \\
 & x_1, x_2 \geq 0
 \end{array}$$

Solution:

Convert to Standard Form (Add Surplus & Artificial Variables):

Constraint	Form	Variables Added
(1)	$\geq$	$-s_1 + a_1$
(2)	$\leq$	$+s_2$
(3)	$\geq$	$-s_3 + a_2$

Standard Form System:

$$\begin{aligned}2x_1 + x_2 - s_1 + a_1 &= 10 \\ -3x_1 + 2x_2 + s_2 &= 6 \\ x_1 + x_2 - s_3 + a_2 &= 6\end{aligned}$$

Objective Function (Big-M form for minimization):

$$Z = 3x_1 + 2x_2 + 0.s_1 + 0.s_2 + 0.s_3 + M.a_1 + M.a_2$$

R code to solve the problem

```
# Load package
# install.packages("lpSolveAPI")

library(lpSolveAPI)

# Define number of variables: x1, x2, s1, s2, s3, a1, a2 = 7 total
lp <- make.lp(0, 7)

# Set objective function: c(x1, x2, s1, s2, s3, a1, a2)
M <- 1e6 # Big M value
set.objfn(lp, c(3, 2, 0, 0, 0, M, M))

# Add constraint 1: 2x1 + x2 - s1 + a1 = 10
add.constraint(lp, c(2, 1, -1, 0, 0, 1, 0), "=", 10)

# Add constraint 2: -3x1 + 2x2 + s2 = 6
add.constraint(lp, c(-3, 2, 0, 1, 0, 0, 0), "=", 6)

# Add constraint 3: x1 + x2 - s3 + a2 = 6
add.constraint(lp, c(1, 1, 0, 0, -1, 0, 1), "=", 6)

# Set variable types (all >= 0)
set.bounds(lp, lower = rep(0, 7), columns = 1:7)

# Set to minimization
lp.control(lp, sense = "min")

# Solve the LP
solve(lp)

# Get solution
solution <- get.variables(lp)
objective <- get.objective(lp)

# Output results
cat("Optimal solution found:\n")
cat("x1 =", solution[1], "\n")
```

```

cat("x2 =", solution[2], "\n")
cat("Minimum Z =", objective, "\n")

##Alternative Solution: (More Easier)

# Load the lpSolve package
library(lpSolve)

# Objective coefficients:  $3x_1 + 2x_2 + M \cdot a_1 + M \cdot a_2$ 
M <- 1e6
obj <- c(3, 2, 0, 0, 0, M, M) # x1, x2, s1, s2, s3, a1, a2

# Constraints matrix
# Each row: coefficients for x1, x2, s1, s2, s3, a1, a2
const_mat <- matrix(c(
  2, 1, -1, 0, 0, 1, 0, # Constraint 1:  $\geq \rightarrow$  surplus & artificial
  -3, 2, 0, 1, 0, 0, 0, # Constraint 2:  $\leq \rightarrow$  slack only
  1, 1, 0, 0, -1, 0, 1 # Constraint 3:  $\geq \rightarrow$  surplus & artificial
), nrow = 3, byrow = TRUE)

# RHS of constraints
const_rhs <- c(10, 6, 6)

# Directions
const_dir <- c("=", "=", "=")

# Solve the LP
solution <- lp(direction = "min",
  objective.in = obj,
  const.mat = const_mat,
  const.dir = const_dir,
  const.rhs = const_rhs)

# Print results
cat("Optimal value of Z:", solution$objval, "\n")
cat("Values of variables (x1, x2):", solution$solution[1:2], "\n")

```

Problem # 07

Solve the following LPP by two-phase method using R code

$$\begin{array}{ll}
 \text{Minimize} & Z = 2x_1 + 3x_2 \\
 \text{Subject to,} & 2x_1 + x_2 \leq 16 \\
 & x_1 + 3x_2 \geq 36 \\
 & x_1 + x_2 = 10 \\
 & x_1, x_2 \geq 0
 \end{array}$$

Solution:

To solve this LPP using the **Two-Phase Simplex Method** in R, you need to:
Convert the problem to **standard form**, adding slack/surplus/artificial variables.

Phase 1: Solve an auxiliary LP to remove artificial variables.

Phase 2: Solve the original LP starting from a feasible solution.

Convert to Standard Form:

Constraint	Form	Variables Added
(1)	$\leq$	$+s_1$
(2)	$\geq$	$-s_2 + a_1$
(3)	$=$	$+a_2$

Variables:

Decision: x_1, x_2

Slack/Surplus: s_1, s_2

Artificial: a_1, a_2

R code:

```
# Load lpSolveAPI package
#install.packages("lpSolveAPI")
library(lpSolveAPI)

# Total variables: x1, x2, s1, s2, a1, a2 => 6 variables
lp <- make.lp(0, 6)

# PHASE 1: Objective = Minimize (a1 + a2)
# Variable order: x1, x2, s1, s2, a1, a2
set.objfn(lp, c(0, 0, 0, 0, 1, 1))

# Constraint 1: 2x1 + x2 + s1 = 16
add.constraint(lp, c(2, 1, 1, 0, 0, 0), "=", 16)

# Constraint 2: x1 + 3x2 - s2 + a1 = 36
add.constraint(lp, c(1, 3, 0, -1, 1, 0), "=", 36)

# Constraint 3: x1 + x2 + a2 = 10
add.constraint(lp, c(1, 1, 0, 0, 0, 1), "=", 10)

# Set minimization direction
lp.control(lp, sense = "min")
```

```

# Solve Phase 1
solve(lp)
sol_phase1 <- get.variables(lp)
cat("Phase 1 solution (artificial objective):", get.objective(lp), "\n")

# Check if artificial variables are zero (feasible)
if (round(sol_phase1[5], 6) == 0 && round(sol_phase1[6], 6) == 0) {
  cat("Feasible solution found. Proceeding to Phase 2...\n")

  # PHASE 2: Original objective function: Min Z = 2x1 + 3x2
  set.objfn(lp, c(2, 3, 0, 0, 0, 0)) # ignore a1, a2 in final objective

  # Solve Phase 2
  solve(lp)
  sol_phase2 <- get.variables(lp)

  cat("Optimal solution:\n")
  cat("x1 =", sol_phase2[1], "\n")
  cat("x2 =", sol_phase2[2], "\n")
  cat("Minimum Z =", get.objective(lp), "\n")
} else {
  cat("No feasible solution exists. Artificial variables are non-zero.\n")
}

```

Explanation:

- `make.lp(0, 6)`: Creates an LP with 0 constraints and 6 variables.
- Variables: x_1 , x_2 , s_1 , s_2 , a_1 , a_2
- **Phase 1** uses auxiliary objective: Minimize $a_1 + a_2$
- If artificial variables are 0 $\rightarrow$ feasible $\rightarrow$ proceed to Phase 2
- **Phase 2** solves original objective using feasible starting point

##Alternative Solution: (More Easier)

```

library(lpsolve)

# ---- PHASE 1 ---- #
# Objective: Minimize a1 + a2 => coefficients for x1,x2,s1,s2,a1,a2
obj_phase1 <- c(0, 0, 0, 0, 1, 1)

# Constraints matrix
# Constraint 1: 2x1 + x2 + s1 = 16
# Constraint 2: x1 + 3x2 - s2 + a1 = 36
# Constraint 3: x1 + x2 + a2 = 10
const_mat1 <- matrix(c(
  2, 1, 1, 0, 0, 0, # ≤ constraint
  1, 3, 0, -1, 1, 0, # ≥ constraint
  1, 1, 0, 0, 0, 1  # = constraint
), nrow = 3, byrow = TRUE)

```

```

const_dir <- c("=", "=", "=")
const_rhs <- c(16, 36, 10)

# Phase 1 LP
phase1 <- lp("min", obj_phase1, const_mat1, const_dir, const_rhs)

cat("Phase 1 (minimize artificial vars):\n")
cat("Optimal value:", phase1$objval, "\n")

# ---- PHASE 2 ---- #
# If feasible (optimum of phase 1 == 0), proceed
if (phase1$objval == 0) {
  # Remove artificial vars (a1, a2) from objective
  obj_phase2 <- c(2, 3, 0, 0, 0, 0)

  # Use same matrix and constraints
  phase2 <- lp("min", obj_phase2, const_mat1, const_dir, const_rhs)

  cat("\nPhase 2 (original objective):\n")
  cat("Optimal value of Z:", phase2$objval, "\n")
  cat("Values of x1, x2:", phase2$solution[1:2], "\n")
} else {
  cat("Problem is infeasible (Phase 1 > 0)\n")
}

```

- **Phase 1** ensures feasibility by driving artificial variables to zero
- **Phase 2** solves the original problem assuming feasible solution exists
- Output shows values of x_1 , x_2 , and minimum Z

Problem # 08

Find the optimum integer solution to the following LPP by Gomory cutting plane method using R code

$$\begin{aligned}
 &\text{Maximize} && Z = x_1 + x_2 \\
 &\text{Subject to,} && 3x_1 + 2x_2 \leq 5 \\
 &&& x_2 \leq 2 \\
 &&& x_1, x_2 \geq 0 \text{ and } x_1, x_2 \text{ are integers.}
 \end{aligned}$$

Solution

```
library(lpsolve)
```

```

# Initial LP data
objective <- c(1, 1)
constraints <- matrix(c(3, 2,

```



```

                                0, 1),
                                nrow = 2, byrow = TRUE)
directions <- c("<=", "<=")
rhs <- c(5, 2)

# Function to check if solution is integer
is_integer_solution <- function(sol, tol = 1e-5) {
  all(abs(sol - round(sol)) < tol)
}

# Start Gomory loop
iteration <- 0
max_iterations <- 10 # safety limit
repeat {
  iteration <- iteration + 1
  cat("\n--- Iteration", iteration, "---\n")

  # Solve current LP
  result <- lp("max", objective, constraints, directions, rhs)

  # Print solution
  cat("x1 =", result$solution[1], "\n")
  cat("x2 =", result$solution[2], "\n")
  cat("Z  =", result$objval, "\n")

  # Check if solution is integer
  if (is_integer_solution(result$solution)) {
    cat("\n✓ Integer solution found.\n")
    break
  }

  # Find most fractional variable
  frac_parts <- result$solution - floor(result$solution)
  most_frac_idx <- which.max(frac_parts)
  if (frac_parts[most_frac_idx] < 1e-5) {
    cat("✗ Rounding error: no significant fractional part.\n")
    break
  }

  # Construct Gomory cut based on current solution
  x1_frac <- frac_parts[1]
  x2_frac <- frac_parts[2]
  cut_coeffs <- c(x1_frac, x2_frac)
  cut_rhs <- floor(sum(cut_coeffs * result$solution))

  # Add Gomory cut
  constraints <- rbind(constraints, cut_coeffs)
  directions <- c(directions, "<=")
  rhs <- c(rhs, cut_rhs)

  if (iteration >= max_iterations) {
    cat("⚠ Max iterations reached. Stopping.\n")
    break
  }
}

```

```
}  
}
```

□ What the Code Does:

Solves the LP relaxation

Checks whether the solution is integer

If not, it adds a Gomory fractional cut

Repeats until an integer solution is found or maximum iterations reached

##Alternative Solution: (More Easier)

```
# Load lpSolve package  
if (!require(lpSolve)) install.packages("lpSolve", dependencies = TRUE)  
library(lpSolve)  
  
# Initial LP (relaxed)  
objective <- c(1, 1) # Coefficients of x1 and x2  
constraints <- matrix(c(3, 2, # 3x1 + 2x2 ≤ 5  
                        0, 1), # x2 ≤ 2  
                      nrow = 2, byrow = TRUE)  
direction <- c("<=", "<=")  
rhs <- c(5, 2)  
  
# Solve relaxed LP  
sol <- lp("max", objective, constraints, direction, rhs, compute.sens = TRUE)  
x <- sol$solution  
cat("Initial LP Solution:\n")  
print(x)  
  
# Check if solution is already integer  
if (all(x == floor(x))) {  
  cat("Optimal Integer Solution Found:\n")  
  cat("x1 =", x[1], "x2 =", x[2], "Max Z =", sol$objval, "\n")  
} else {  
  # Add integer constraints and solve directly  
  int_sol <- lp("max", objective, constraints, direction, rhs,  
               all.int = TRUE)  
  x_int <- int_sol$solution  
  cat("Integer Solution (via Gomory cuts):\n")  
  cat("x1 =", x_int[1], "x2 =", x_int[2], "Max Z =", int_sol$objval, "\n")  
}
```

Problem # 09

Find the initial basic feasible solution for the following TP using North-West Corner Rule (NWCR) and Vogel's Approximation Method (VAM) using R code.

Destination Source	D_1	D_2	D_3	Supply
S_1	4	8	8	76
S_2	16	24	16	82
S_3	8	16	24	77
Demand	72	102	41	

Solution:

```
# Define the cost matrix
costs <- matrix(c(4, 8, 8,
                  16, 24, 16,
                  8, 16, 24),
               nrow = 3, byrow = TRUE)

supply <- c(76, 82, 77)
demand <- c(72, 102, 41)

# Function for North-West Corner Rule
north_west_corner <- function(supply, demand) {
  m <- length(supply)
  n <- length(demand)
  allocation <- matrix(0, m, n)

  i <- 1
  j <- 1
  while (i <= m & j <= n) {
    alloc <- min(supply[i], demand[j])
    allocation[i, j] <- alloc
    supply[i] <- supply[i] - alloc
    demand[j] <- demand[j] - alloc

    if (supply[i] == 0) {
      i <- i + 1
    } else {
      j <- j + 1
    }
  }
  return(allocation)
}

# Function for Vogel's Approximation Method (VAM)
vogel_approximation <- function(costs, supply, demand) {
  m <- nrow(costs)
  n <- ncol(costs)
  allocation <- matrix(0, m, n)
```

```

supply_left <- supply
demand_left <- demand
cost_matrix <- costs

while (any(supply_left > 0) && any(demand_left > 0)) {
  penalty <- rep(0, m + n)

  # Calculate row penalties
  for (i in 1:m) {
    if (supply_left[i] > 0) {
      row <- cost_matrix[i, demand_left > 0]
      if (length(row) >= 2) {
        penalty[i] <- sort(row)[2] - sort(row)[1]
      } else if (length(row) == 1) {
        penalty[i] <- row
      }
    }
  }

  # Calculate column penalties
  for (j in 1:n) {
    if (demand_left[j] > 0) {
      col <- cost_matrix[supply_left > 0, j]
      if (length(col) >= 2) {
        penalty[m + j] <- sort(col)[2] - sort(col)[1]
      } else if (length(col) == 1) {
        penalty[m + j] <- col
      }
    }
  }

  # Find max penalty
  idx <- which.max(penalty)

  if (idx <= m) {
    # Row penalty selected
    i <- idx
    available_cols <- which(demand_left > 0)
    j <- available_cols[which.min(cost_matrix[i, available_cols])]
  } else {
    # Column penalty selected
    j <- idx - m
    available_rows <- which(supply_left > 0)
    i <- available_rows[which.min(cost_matrix[available_rows, j])]
  }

  alloc <- min(supply_left[i], demand_left[j])
  allocation[i, j] <- alloc
  supply_left[i] <- supply_left[i] - alloc
  demand_left[j] <- demand_left[j] - alloc
}
return(allocation)
}

```

```
# Run NWCR
nwcr_result <- north_west_corner(supply, demand)
cat("North-West Corner Rule Allocation:\n")
print(nwcr_result)

# Run VAM
vam_result <- vogel_approximation(costs, supply, demand)
cat("\nVogel's Approximation Method Allocation:\n")
print(vam_result)
```